

ИССЛЕДОВАНИЕ

21 ЯНВАРЯ 2026 ГОДА

Представляем RadixMLP: внутрипакетную дедупликацию для причинно-следственных преобразователей

RadixMLP устраняет избыточные вычисления с использованием общих префиксов, что позволяет ускорить работу с реальными задачами переранжирования в 1,4–1,6 раза, а с синтетическими тестами — до 5 раз.



Майкл Фейл

ЧИТАТЬ БУМАГУ

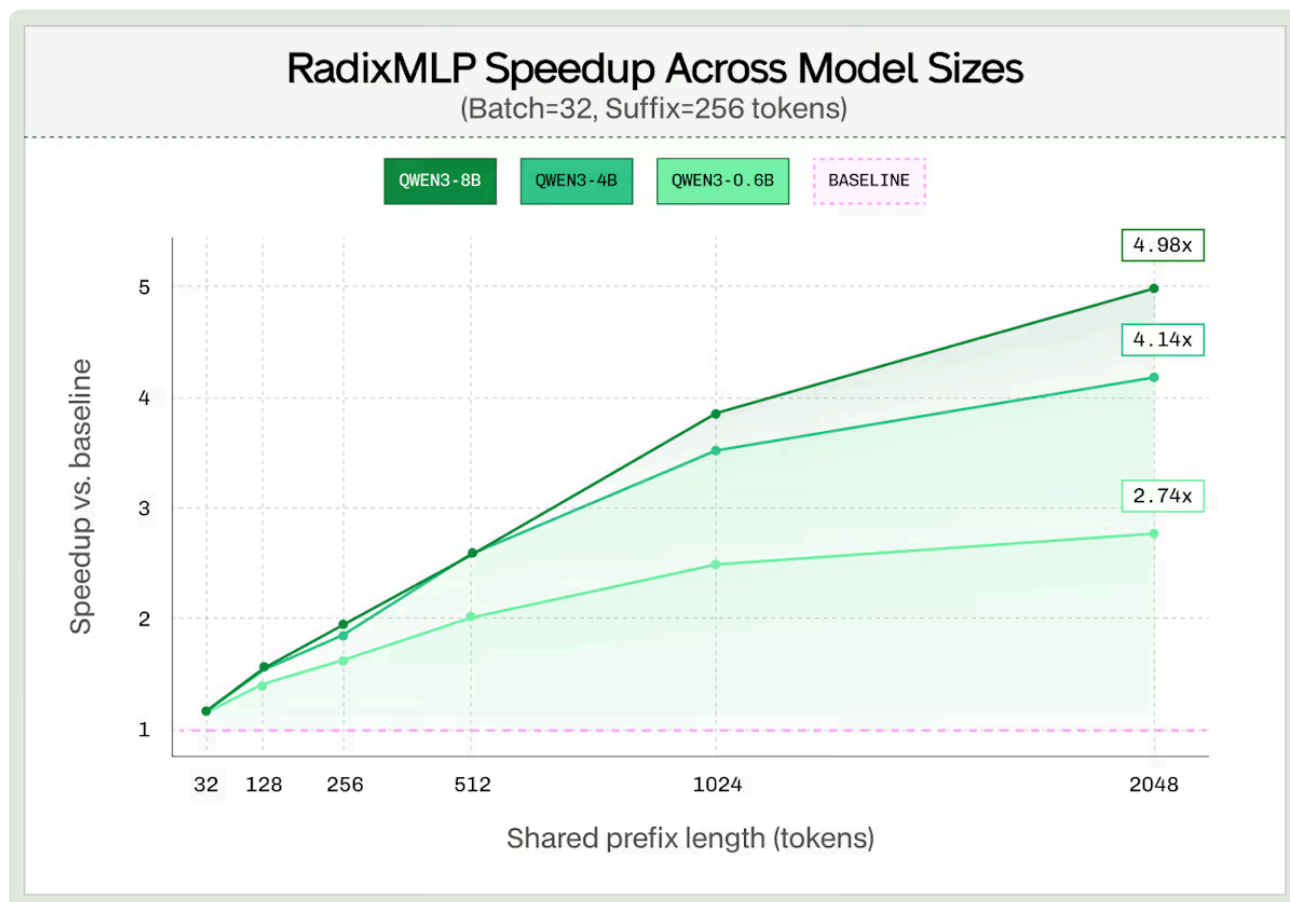
При пакетной обработке данных для моделей с причинно-следственными связями часто обрабатываются последовательности с общими префиксами, например системные подсказки, примеры с малым количеством выборок или общие запросы в задачах повторного ранжирования. Стандартные механизмы логического вывода обрабатывают каждую последовательность независимо друг от друга, повторно вычисляя идентичные активации многослойного перцептрона для каждой копии общего префикса.

Чтобы устранить эту избыточность, мы разработали **RadixMLP** — метод, использующий позиционную природу многослойных свёрточных нейронных сетей, слоевых норм, линейных проекций и эмбедингов. RadixMLP динамически сопоставляет пакеты данных с префиксным деревом, объединяя общие сегменты в сжатое представление для позиционных вычислений и распределяя результаты только на границах внимания.

В комплексных тестах на [MS MARCO v1.1](#) с моделями Qwen3 (от 0,6 до 8 миллиардов параметров) RadixMLP обеспечивает **ускорение в 1,44–1,59 раза** при реалистичных

сценариях повторного ранжирования и до **ускорения в 5 раз** при синтетических тестах с более длинными общими префиксами.

RadixMLP имеет открытый исходный код и готов к использованию в продакшене: вы можете использовать его уже сегодня на Baseten в рамках Baseten Embeddings Inference (BEI — пример конфигурации ниже), а также в TEI. Свяжитесь с нами, если у вас есть вопросы.



На синтетических тестах с более длинными общими префиксами мы наблюдаем ускорение в 2,74–4,98 раза при использовании RadixMLP с моделями Qwen разного размера.

Проблема: избыточные вычисления при пакетном логическом выводе

Рассмотрим типичную рабочую нагрузку при пакетном логическом выводе:

- **Встраиваемые модели** добавляют один и тот же глобальный контекст к каждому документу.
- **Перекрестные кодировщики** сопоставляют каждый фрагмент текста с одним и тем же запросом.
- **Системы классификации** используют одни и те же примеры с малым количеством данных для разных входных данных.

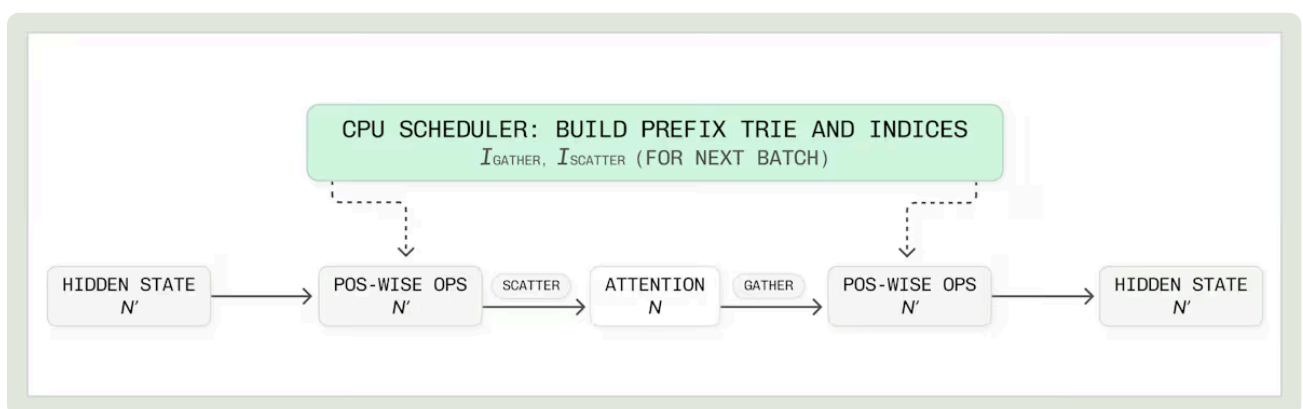
Даже при использовании рваных макетов, исключая отступы, повторяющиеся токены с префиксом по-прежнему материализуются в виде отдельных строк в памяти графического процессора. Таким образом, модель пересчитывает идентичные активации для каждой копии. Эта неэффективность особенно заметна на этапе предварительного заполнения, когда позиционные компоненты (многослойные нейронные сети, LayerNorm и линейные проекции) составляют значительную часть от общего количества операций с плавающей запятой.

Это наблюдение легло в основу **RadixMLP** — метода, призванного устранить избыточные вычисления префиксов за счет использования позиционной структуры слоев трансформера.

RadixMLP: подробный технический разбор

В то время как причинно-следственное самовнимание представляет собой операцию смешивания последовательностей, многослойный перцептрон, LayerNorm, линейные проекции (Q, K, V, O) и эмбединги применяются попозиционно. Для токенов с одинаковой причинно-следственной историей (то есть с одинаковым префиксом) эти вычисления для каждого токена идентичны и могут быть использованы повторно.

Эта инвариантность сохраняется на всех уровнях: если у токенов общий префикс, их выходные данные по позициям остаются одинаковыми на каждом уровне. Поскольку внимание к токенам с общим префиксом зависит только от предыдущих токенов с тем же префиксом, их выходные данные также идентичны. Таким образом, общее представление сохраняется на всех уровнях.



Интеграция RadixMLP в каузальные преобразователи. Скрытое состояние остается в компактном пространстве (N' токенов) между слоями. Позиционные операции (нормализация перед вниманием, проекции, MLP) выполняются в компактном пространстве. Только операция внимания требует исходного пространства (N токенов), а после внимания выполняются операции сбора и распределения.

Конструкция префиксного дерева

RadixMLP строит префиксное дерево на основе пакета данных. Каждый узел дерева идентифицируется по родительскому узлу и следующей `(token_id, position_id)` паре, поэтому узлы привязаны к конкретному пути. Не требуется хранить древовидную структуру на графическом процессоре. На практике мы материализуем только результирующие индексные карты (`I_gather`, `I_scatter`) для выполнения на графическом процессоре.

For a batch of B sequences with shared prefix length P and per-sequence suffix length S , the compression ratio is:

$$r = \frac{N}{N'} = \frac{B(P+S)}{P+BS}$$

Where N is the total number of tokens and N' is the number of compacted tokens. In batch workloads with shared prefixes, typically $N' \ll N$.

Gather and scatter operations

We define two index mapping tensors:

1. **Gather indices** (`I_gather` $\in \mathbb{N}^{N'}$): For each position in the compact buffer, this specifies which position in the original input to read and selects a representative occurrence for that compact token.
2. **Scatter indices** (`I_scatter` $\in \mathbb{N}^N$): For each position in the original layout, this specifies which position in the compact result buffer to read and broadcasts computed results back to all duplicate positions.

The forward pass for the MLP block is modified as follows:

```
X_unique = X[I_gather]
Y_unique = MLP(X_unique)
Y_restored = Y_unique[I_scatter]
```

Since the MLP has complexity $O(Nd^2)$, the cost is reduced to $O(N'd^2)$. As the gather/scatter operations are memory-bound $O(N)$ copies, the net speedup approaches the compression ratio $r = N/N'$ as the hidden dimension d grows.

Integration with attention

A critical requirement is **causal consistency**. While position-wise layers are independent per token, the attention mechanism requires the full context window.

We place **scatter** and **gather** operations at the boundary between position-wise and sequence-mixing computations:

1. Compute all position-wise operations in compact space: pre-attention LayerNorm, RoPE, Embedding Layer, Q/K norms, and the Q, K, V projections on N' tokens
2. Scatter the resulting Q/K/V tensors back to the full N -token ragged layout
3. Run the attention operation (FlashAttention) in full space
4. Gather the attention output back to a compact space for the O projection, post-attention LayerNorm, MLP, and residual additions

Position indices are preserved alongside compact tokens: we gather the position ID tensor using `I_gather`, so each compact token retains its original sequence position for RoPE computation.

Key innovation: Stateless design

Unlike PagedAttention or RadixAttention, which require persistent KV caches with block tables, eviction policies, and distributed coordination, **RadixMLP operates entirely within a single forward pass.**

Beyond that, it's the first cache-like operator for training and deduplicating tokens based on common prefixes. This makes it valuable for batch inference, where maintaining caches for millions of heterogeneous documents is impractical. It enables libraries like [Baseten Embeddings Inference \(BEI\)](#) to achieve cache-like speedups, even when inference times are in the 5-millisecond range.

CPU-side scheduling

Constructing the trie and generating index maps on GPUs would introduce synchronization overheads. Instead, we utilize a **pipelined scheduler**:

- While the GPU executes batch t , the CPU scheduler analyzes the request queue for batch $t+1$
- The scheduler constructs the prefix trie and pre-calculates `I_gather` and `I_scatter`

With an efficient Rust implementation, computing approximately 16,384 tokens takes between 129 μ s and 750 μ s on a single-thread Intel Xeon CPU (depending on the prefix ratio). This is around **3 orders of magnitude below the corresponding GPU inference time**, making even synchronous usage feasible.

Results

MS MARCO v1.1 benchmarks

We evaluated a real serving pipeline by benchmarking TEI on MS MARCO v1.1 validation query–passage pairs. Each request embeds multiple query–document pairs formatted with a reranker-style Qwen3 chat template.

Model	Base 50 (s)	Radix P50 (s)	Speedup
Qwen3-0.6B	0.78	0.54	1.44x
Qwen3-4B	3.76	2.42	1.56x
Qwen3-8B	5.96	3.74	1.59x

These gains are smaller than synthetic forward-pass speedups because prefix sharing is less extreme in this workload and because end-to-end latency includes non-model overheads. Still, the improvement remains substantial in a practical serving setting.

Synthetic benchmarks

For controlled evaluation, we used a synthetic fixed batch size of **B=32** sequences, varying the prefix length (shared across all sequences) and suffix length (unique per sequence).

Configuration	Model	Speedup
2048/256	Qwen3-0.6B	2.74x
2048/256	Qwen3-4B	4.14x
2048/256	Qwen3-8B	4.98x

We define the compact-token ratio $\gamma = \frac{N'}{N} \in (0, 1]$ and the corresponding compression ratio $r = \frac{N}{N'} = \frac{1}{\gamma}$. The results demonstrate three key trends:

- **Prefix length:** Longer shared prefixes yield lower γ (higher r) and greater speedups

- **Model size:** Larger models benefit more from RadixMLP, as MLP compute dominates over gather/scatter overhead
- **Suffix ratio:** Shorter suffixes (higher prefix:suffix ratio) maximize compression benefits

Comparison to vLLM

We compared TEI with and without RadixMLP against vLLM v0.13.0 on two MS MARCO configurations:

Model	γ	TEI	TEI RadixMLP	vLLM
Qwen3-0.6B	0.61	0.78	0.55	0.71

TEI without RadixMLP is the worst-performing inference system. With RadixMLP enabled, TEI outperforms vLLM for the Qwen3-0.6B model. For larger models, vLLM outperforms TEI by a 3-7% margin.

A key difference is memory usage: TEI uses ~17GB vs ~79GB for vLLM with Qwen3-8B, as no additional memory is allocated for a paged KV cache.

Implementation: Deploying RadixMLP with Truss

Here's how to deploy a Qwen3 model with RadixMLP using Truss:

```
1 # Qwen3 with RadixMLP
2
3 model_name: BEI-qwen3-radixmlp
4 python_version: py39
5
6 model_metadata:
7   example_model_input:
8     encoding_format: float
9     input:
10       type: string
11     model: model
12
13 resources:
14   accelerator: H100
15   cpu: 1
16   memory: 10Gi
17   use_gpu: true
```

```
18
19 trt_llm:
20     build:
21         base_model: encoder_bert
22         checkpoint_repository:
23             repo: michaelfeil/Qwen3-Embedding-0.6B-auto
24             revision: main
25             source: HF
26         max_num_tokens: 32768
27     runtime:
28         webserver_default_route: /v1/embeddings
```

Deploy with:

```
truss push --publish
```

When to use RadixMLP

RadixMLP excels in scenarios with high prefix redundancy in prefill-heavy workloads. Some such workloads are often:

- **Reranking:** Query-passage pairs with shared system prompts
- **Embeddings:** Global context prepended to every document
- **Post-Training:** Shared system templates in RLHF
- **Classification:** Few-shot examples reused across inputs
- **Instruction following:** Shared instruction templates

Related work

RadixMLP operates orthogonally to several existing optimization techniques that are utilized by Baseten today:

- **PagedAttention:** Introduces paged, block-based KV management with block-level sharing. These stateful approaches require complex scheduling and are limited by block granularity, whereas RadixMLP offers block-free, stateless reuse within individual batches.

- **RadixAttention:** Organizes prefixes in a radix tree for cross-request KV reuse. Requires persistent state management, while RadixMLP operates within a single forward pass.
- **FlashInfer Cascade Inference:** Computes attention over shared prefix segments once, then merges partial attention states with per-request suffix attention. RadixMLP complements this by optimizing position-wise components. This is similar to HydraGen kernels.

These approaches are fully complementary—RadixMLP can stack with KV cache, attention optimizations, or batching/scheduling optimizations.

Training compatibility

While the focus of this work is inference, RadixMLP is compatible with training because the compaction operators are differentiable. The compacting gather is an `index_select`, and its backward pass is a scatter-add into the original layout (`index_add`), accumulating gradients when multiple original tokens map to the same compact representative.

We validated both forward and backward correctness on a 2-layer Qwen3-style model across synthetic prefix-sharing patterns. With PyTorch SDPA attention as the reference implementation, enabling RadixMLP produces numerically identical logits up to the specified tolerances (`rtol=1e-4` , `atol=1e-4`) and gradient differences on the order of 10^{-5} or lower.

RadixMLP is open-source

RadixMLP and Index-Select Kernels, as well as the experimental results, are open-source under the MIT License. You can find the GitHub repository [here](#).

You can read the full paper on arXiv [here](#). [Reach out](#) to talk to our engineers about optimizing model performance for your workloads.

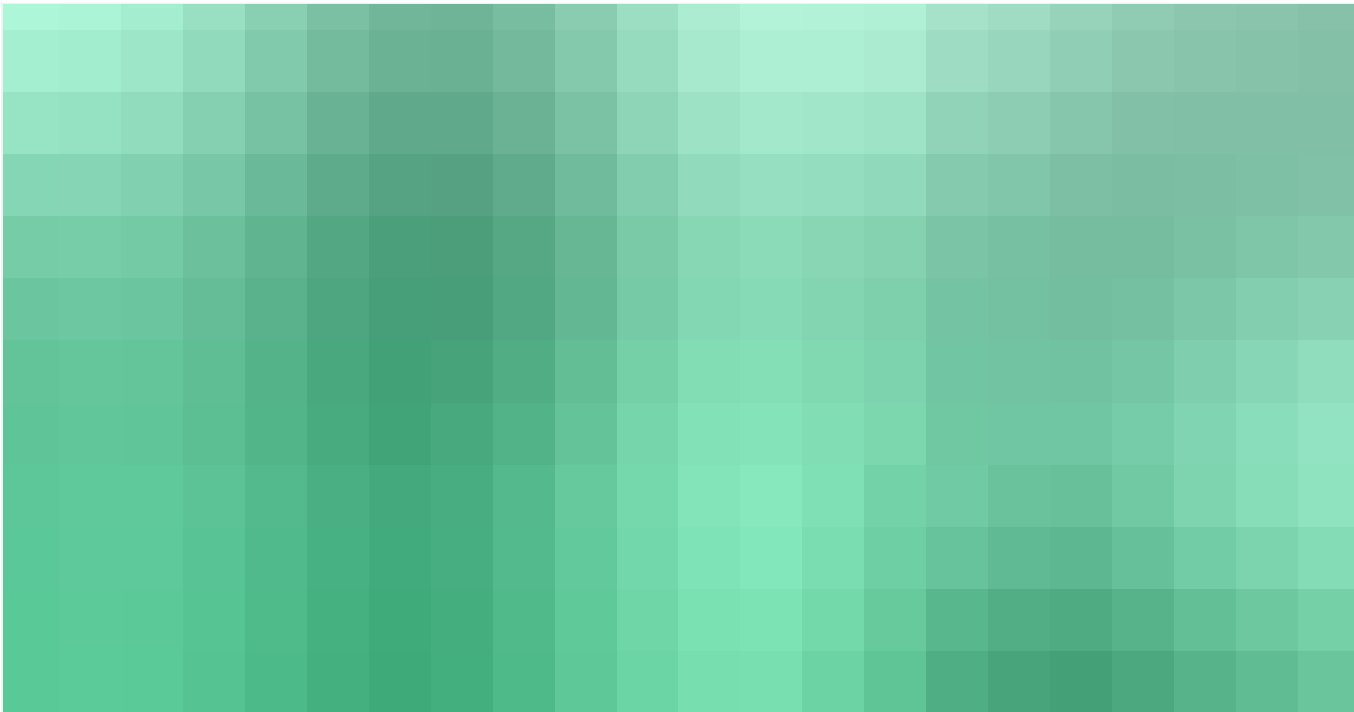
For attribution in academic contexts, please cite this work as:

Feil, Michael. "Introducing RadixMLP: Intra-Batch Deduplication for Causal Transformers." arXiv, January 21, 2026. <https://arxiv.org/abs/2601.15013>

BibTeX citation

```
@misc{feil2026radixmlp,  
  author      = {Feil, Michael},  
  title       = {Introducing {RadixMLP}: Intra-Batch Deduplication for Causal  
Transformers},  
  year        = {2026},  
  month       = jan,  
  day         = {21},  
  howpublished = {arXiv},  
  url         = {https://arxiv.org/abs/2601.15013},  
}
```

Other research



RESEARCH

Post-Training Science for Supervised Fine-Tuning

How the optimal learning rate and batch size move with model scale, family, and data, and whether one selection rule transfers across them.



CHARLES O'NEILL • 2 others



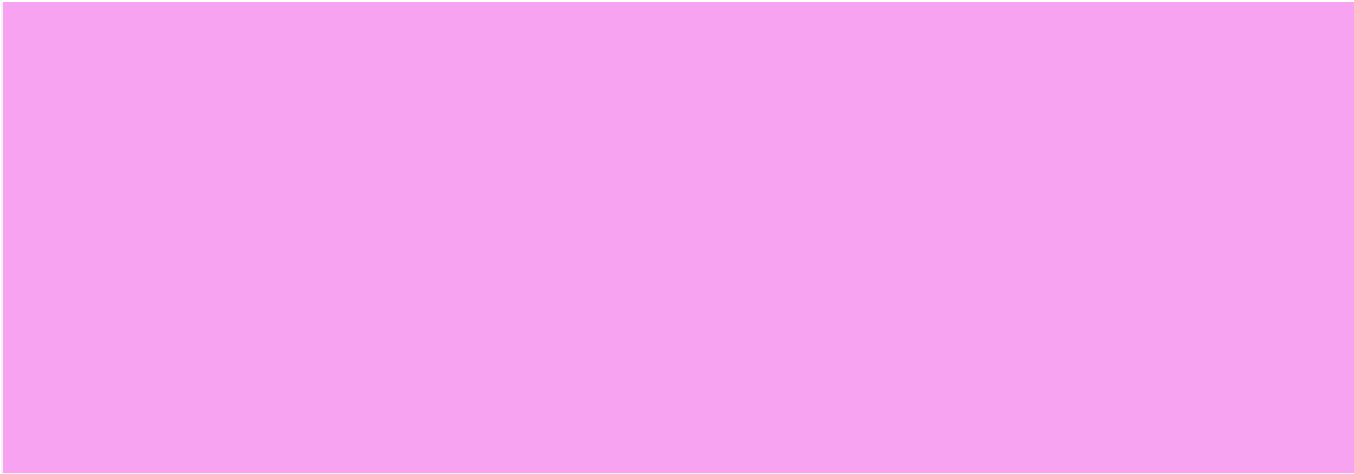
RESEARCH

Still: Amortized KV Cache Compaction in a Single Forward Pass

You can shrink a language model's KV cache by 200×, in a single forward pass, and it still answers correctly.



CHARLES O'NEILL • 4 others



Explore Baseten today

START DEPLOYING

TALK TO AN ENGINEER >

Product

Dedicated Inference

Model APIs

Training

Frontier Gateway

Baseten Inference Platform

Model Runtimes

Infrastructure

Multi-cloud

Deployment options

Cloud

Self-hosted

Hybrid

Embedded engineering

Forward deployed engineers

Modalities

Transcription

Image Generation

Text-to-speech

Large language models

Compound AI

Embeddings

Industries

Enterprise

Healthcare

Developer

Model library

Documentation

Changelog

Resources

Research

Customers

Blog

Guides

Events

Partners

Savings calculator

Trust Center

About us

Startups

Careers

Contact us

Popular models

GLM 5.2

Kimi K2.7 Code

DeepSeek V4

GPT OSS 120B

Whisper Large V3

NVIDIA Nemotron 3 Ultra

Explore all

Legal

Terms and Conditions

Privacy Policy

Service Level Agreement

● ALL SYSTEMS NORMAL



